

Build and Verify Your Local AI Workbench

Week 2 · Computing Commons

The finish line is not “it is installed.”

The finish line is evidence that the workflow works.

Today's finish line

DO THIS Trace one request through the local stack.

WHAT THIS MEANS
Each layer has a different job.

DO THIS Make one tiny, bounded change.

WHAT THIS MEANS
Aider can change files; you still inspect them.

DO THIS Prove the result independently.

WHAT THIS MEANS
A diff shows change. A test checks behavior.

What does this prove, and what does it NOT prove?

DO THIS → WHAT THIS MEANS → FIND THIS YOURSELF → DEMAND DOCUMENTATION → VERIFY ON YOUR MACHINE →
KEEP THE RECEIPT

Why these tools? The workbench is infrastructure

- ▶ Computing work means files, programs, debugging, documentation, tests, and source history.
- ▶ The tools are a shared workbench—not a random software collection.
- ▶ You remain responsible for intent, scope, judgment, and the conclusion.

Our rule

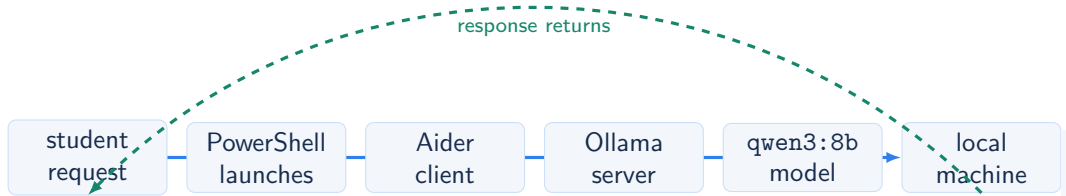
Tools can move quickly.
Evidence must move with them.

Humans provide meaning.

Machines provide motion.

Primary source: Computing Commons AGENTS.md; SWOSU curriculum philosophy

The stack: layers, not magic



DO THIS → WHAT THIS MEANS → FIND THIS YOURSELF → DEMAND DOCUMENTATION → VERIFY ON YOUR MACHINE →
KEEP THE RECEIPT

Evidence has layers too

Observation	Supports	Does not support
<code>python -version</code>	Python command resolves	Your program is correct
<code>ollama list</code>	Exact model is reported available	The model can answer
Loopback API check	Local service answered	Aider is integrated
Direct inference	One model request completed	The answer is correct
<code>git diff</code>	What text changed	Behavior is correct
Independent test	Tested behavior passed	Every behavior passed

Installed · listed · open · generated are observations, not conclusions.

First: open PowerShell

1. Click Windows **Start**.
2. Type **PowerShell**.
3. Select the provisioned **PowerShell 7** window.
4. Click inside; look for a prompt ending in **>**.

```
PS C:\Users\Student>
```

This is the prompt. It tells you where the command will run.

Do not paste a command until you can point to the prompt.

FIND THIS YOURSELF: “Windows 11 + PowerShell 7: how do I verify which version is actually running? Give the exact command, official Microsoft documentation, and limits.”

Verify PowerShell itself

DO THIS At the prompt, run: `$PSVersionTable.PSVersion`

WHAT THIS MEANS PowerShell is the command environment used to navigate folders, launch tools, and repeat checks.

DEMAND DOCUMENTATION

Find the official Microsoft page for `PSVersionTable`; check platform relevance.

VERIFY ON YOUR MACHINE

Read the version printed by your own terminal. Do not substitute a screenshot.

KEEP THE RECEIPT Save the command, timestamp, and smallest useful output.

Python: the runtime for our program

DO THIS Run: `python
-version`

WHAT THIS MEANS Python
executes the supplied exercise and
its tests.

What this proves

- ▶ The command resolves.
- ▶ A runtime reports a version.

What it does not prove

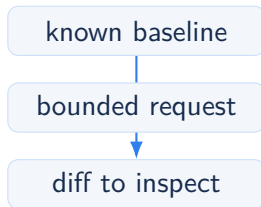
- ▶ Packages are present.
- ▶ Your project or test is correct.

FIND THIS YOURSELF: “Official Python documentation: how do I check the version on Windows, and what does that check leave untested?”

Git: the before-and-after witness

DO THIS Inside the supplied disposable exercise, run: `git status` and `git diff`

WHAT THIS MEANS Git exposes the starting state and the exact file change.

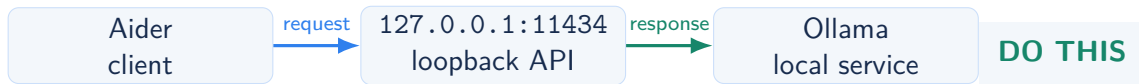


Ask: Which line changed? Which file?
Any extras?

DEMAND DOCUMENTATION: official Git docs for `status` and `diff`.

KEEP THE RECEIPT: the diff, not the model's claim.

Ollama: a local service on loopback



DEMAND DOCUMENTATION: Use Ollama's official API documentation; ask what endpoint and response were observed.

The exact approved model: qwen3:8b

DO THIS Check the exact tag:
ollama list

Then ask: Did a real request
complete?

LISTED

model is reported
available**INFERENCE**

model actually answered one request

VERIFY ON YOUR MACHINE: Send the small approved direct request.

KEEP THE RECEIPT Exact model tag, command, response, exit status.

Aider is the coding client, not the model

- ▶ Works from the project folder.
- ▶ Gathers bounded file context.
- ▶ Communicates with the selected model through Ollama.
- ▶ Leaves changes for you to review.

```
aider -version
```

Proves: command reports a version.

Does not prove: local Ollama configuration, safe scope, or correct code.

BOUNDARY: approved profile → ollama_chat/qwen3:8b →
http://127.0.0.1:11434, num_ctx: 8192. No cloud key or provider detour.

The first Aider win: one bounded change

DO THIS Start from the supplied disposable Git exercise. Record baseline. Launch the approved local profile. Ask for exactly one narrow change.

baseline → ask → inspect → diff → test

A good request

“Change the supplied function so the expected test passes. Edit only the implementation file. Do not change tests.”

A bad request

“Fix everything, improve the project, and make it production-ready.”

Scope is a safety feature.

Inspect the diff before you trust it

DO THIS Exit Aider, then run the approved Diff command.

Look for:

- ▶ intended file only;
- ▶ intended line(s) only;
- ▶ no secret, test, or unrelated edits;
- ▶ a change you can explain.

“Aider changed it” is not “the change is correct.”

Independent test: a different claim

DO THIS Run the supplied independent final test.

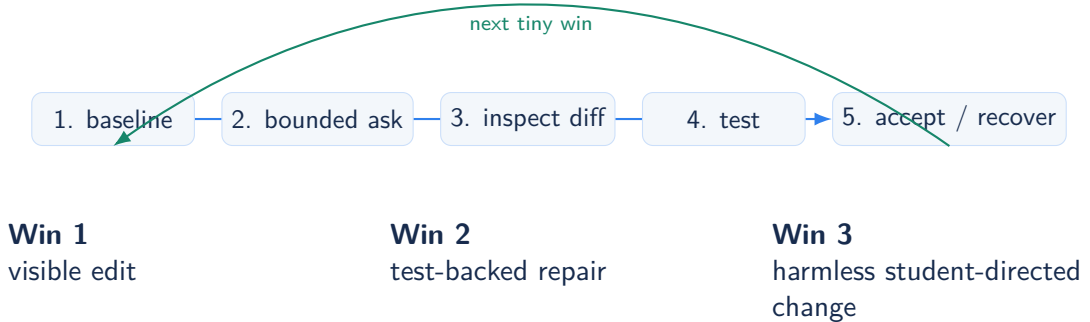
WHAT THIS MEANS A test reruns a defined behavior without trusting the model's narration.

Passing supports: the behavior this test checks.

Not proof of: every input, every file, or every possible property.

KEEP THE RECEIPT Preserve exit status and smallest useful test output.

Three tiny wins: repeat the loop



Each loop earns a little more independence.

“NOT READY”

is useful evidence.

Stop at the first failed check.

Record the claim, command, exit status, smallest output, and diagnostic code.

One bounded retry

1. Confirm the supplied folder.
2. Run the same documented check once.
3. Escalate if it still fails.

Do not: switch models, scan ports, add credentials, download, elevate, or repeatedly repair services.

Recitation is the recovery seam—not a reason to improvise infrastructure.

Prompt structure: from answer to evidence

WEAK

“How do I check PowerShell?”

BETTER

“Windows 11 + PowerShell 7: give exact command and expected output.”

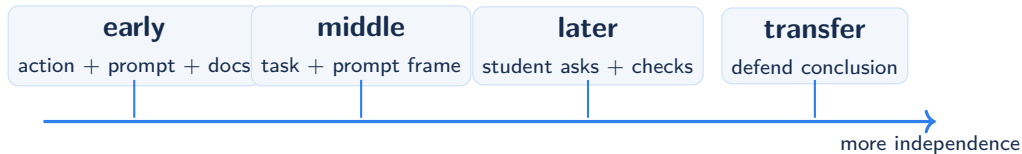
VERIFICATION-ORIENTED

“Cite Microsoft docs; say what it proves and does not prove; give an independent check.”

Do not stop at the answer. Ask how to verify it.

FIND THIS YOURSELF: build the question with platform, version, action, observation, primary source, and limits.

Learning to fish: the scaffold fades



Your job changes: from following a recipe to choosing a defensible check.

The habit stays: claim → observation
→ limit → conclusion.

DO THIS → WHAT THIS MEANS → FIND THIS YOURSELF → DEMAND DOCUMENTATION → VERIFY ON YOUR MACHINE →
KEEP THE RECEIPT

Work First: the bridge



This week is the small, teachable version of a professional loop.

Show that it works: the Week 2 receipt

By the end, demonstrate:

- ▶ PowerShell and foundation checks;
- ▶ direct Ollama success with exact `qwen3:8b`;
- ▶ one bounded Aider change;
- ▶ inspected `git diff`;
- ▶ independent test result;
- ▶ one sentence explaining the loop.

Receipt

Claim · command · exit status · smallest output · conclusion · limit

If not ready

Preserve failure evidence and use Recitation/support.

A small receipt beats a confident story.

Your turn

Ask narrowly. Inspect honestly. Test independently. Keep the receipt.

What does this prove?